

Assignment 3: Bridge Pattern

Student: Akmarzhan Nurtas

Group: Se-2538

Topic: A — Drawing Shapes

Submitted commit: c966a65

Base commit: db40973

1. Why Bridge for this problem

We had to pick one topic out of A–D. I chose A because the two dimensions are easy to see: the shape can change, and the way we draw it can change, and neither depends on the other.

The two dimensions are:

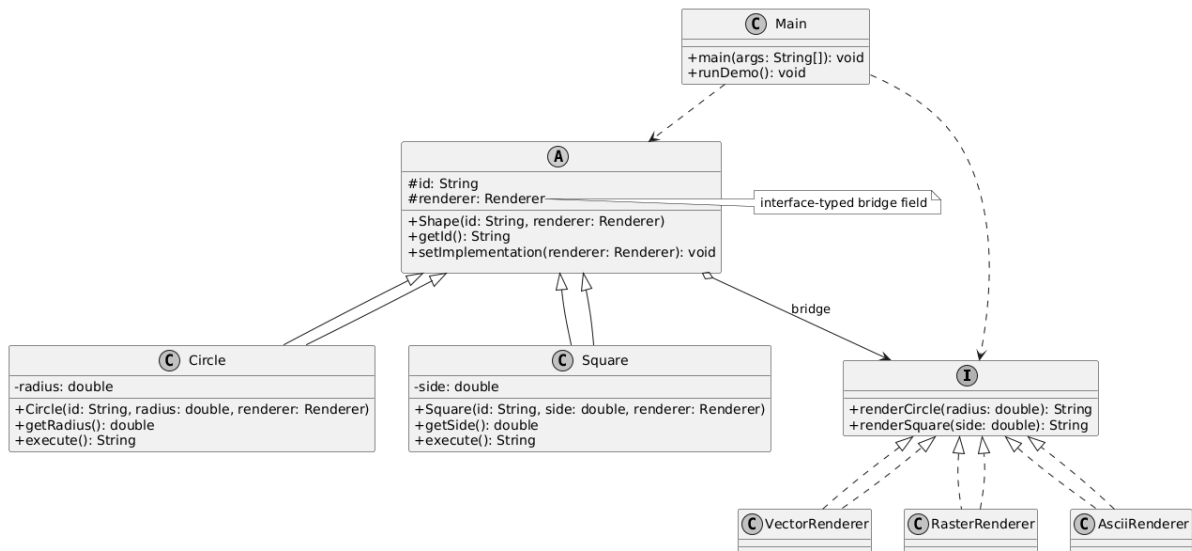
- Shape type: Circle, Square
- Rendering type: Vector, Raster, Ascii

If I didn't use Bridge, I'd have to write six classes: VectorCircle, RasterCircle, AsciiCircle, VectorSquare, RasterSquare, AsciiSquare. That's 2×3 . Every time I add a shape, I'd have to write three more classes. Every time I add a renderer, I'd double the whole thing.

Bridge avoids this. Shape holds a Renderer reference. Circle and Square only know their own data (radius, side) and don't care how they are drawn. Vector/Raster/Ascii know how to draw but don't care what shape they're drawing. When I added AsciiRenderer later, I only wrote one class — not one new class for each shape.

2. UML diagram

The left side is the abstraction: Shape with Circle and Square under it. The right side is the implementor: Renderer with three classes. The arrow from Shape to Renderer is the bridge — just a field. Main uses Shape and Renderer, nothing else.



3. Code examples

(1) Shape keeps only identity and delegation.

```
public abstract class Shape {
    protected final String id;
    protected Renderer renderer;
    protected Shape(String id, Renderer renderer) {
        this.id = id;
        this.renderer = renderer;
    }
    public String getId() {
        return id;
    }
    public void setImplementation(Renderer renderer) {
        this.renderer = renderer;
    }
    public abstract String execute();
}
```

Shape doesn't know how to draw anything. It only stores the id and the renderer, and it exposes a setter so the renderer can be swapped. That's it. The abstract execute() is the one thing subclasses have to fill in.

(2) Names say what they do.

```
@Override
public String execute() {
    return renderer.renderCircle(radius);
}
```

execute, renderCircle, radius — I don't need a comment. Someone reading this knows exactly what happens.

(3) Each method does one thing.

```
@Override
public String renderSquare(double side) {
    return "VECTOR square side=" + side;
}
```

One line. No branching, no state changes. Easy to test.

(4) No duplicated workflow.

Circle and Square have almost the same execute() shape:

```
return renderer.renderCircle(radius);
return renderer.renderSquare(side)
```

The common part — "call the renderer and return its answer" — is not duplicated anywhere. Each subclass only says which renderer method to call. The shared part lives in Shape as the abstract execute() contract.

(5) State is private.

```
private final double radius;

public double getradius() {
    return radius;
}
```

Nobody outside Circle can change radius. Same for id in Shape (protected final). This keeps the abstraction-side data safe from the client and from the renderers.

4. Adding I3 later

After the base version (I1 and I2) was working and committed, I added AsciiRenderer in a second commit. The diff between the two commits contains only:

- new file src/AsciiRenderer.java
- src/Main.java — added T6 and T7
- sources.txt — added the new path

Shape, Circle, Square, Renderer, VectorRenderer, RasterRenderer were not edited at all. This is the real point of Bridge — I could extend one side without touching the other.

A trade-off. Bridge adds one extra layer. execute() doesn't build the string itself, it asks another object. For a very small program this is more classes than needed. But for this assignment the point is exactly to be able to swap the renderer at runtime — that's what T5

shows. Without the bridge I couldn't do `shape.setImplementation(new RasterRenderer())` on the same object.

Bridge vs Adapter. They look similar — both use an interface and a wrapper. The difference is when and why. Adapter is a fix: you already have a class with the wrong interface, and you wrap it so the client can use it. Bridge is a design decision: you split the abstraction and the implementation from the start so they can grow independently. In my code the `Renderer` interface is not a workaround for something that already exists — it's the second axis by design.

5. Sources

Lecture 4 (Bridge Pattern), Software Design Pattern

- Moodle Bridge reading
- Gamma et al., Design Patterns, 1994